Práctica s09: Mexi-Banco (Mensajería Asíncrona y Resiliencia)

Alumno: Diego Alexander Martínez

1. Arquitectura de Microservicios y Contenedorización

Objetivo: Desacoplar el sistema bancario monolítico en servicios independientes para mejorar la escalabilidad y el mantenimiento, desplegando la infraestructura mediante contenedores.

Desarrollo y Resultados: Se implementó una arquitectura orientada a microservicios dividiendo el dominio del banco en módulos independientes: cuenta, transferencia, notificacion y un gateway para enrutar las peticiones. Cada microservicio fue empaquetado en su propio contenedor utilizando Docker y cuenta con su propia base de datos PostgreSQL, garantizando el aislamiento de datos (Database-per-service). La orquestación de todos los contenedores, incluyendo las bases de datos y el broker de mensajería, se gestionó a través de un archivo docker-compose.yml, permitiendo levantar todo el entorno de desarrollo con un solo comando.

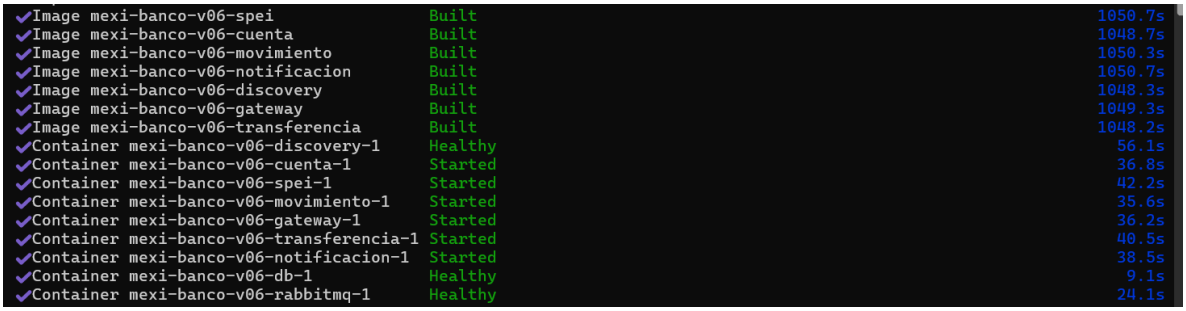


Figura 1: Despliegue de los microservicios y bases de datos independientes mediante Docker Compose.

2. Integración de Mensajería Asíncrona con RabbitMQ

Objetivo: Eliminar el acoplamiento temporal e invocaciones HTTP síncronas entre los servicios mediante la introducción de un Message Broker.

Desarrollo y Resultados: Para que los servicios se comuniquen sin depender de que el otro esté disponible en ese exacto milisegundo, se introdujo RabbitMQ. Se configuró un *Topic Exchange* llamado transferencias. Ahora, cuando el servicio de transferencia procesa un movimiento, ya no llama directamente por HTTP al servicio de notificacion. En su lugar, publica un evento (ej. transferencia.realizada) en el exchange. El servicio de notificacion

actúa como un consumidor, suscrito a la cola correspondiente, procesando los avisos en segundo plano. Esto aumentó la tolerancia a fallos del sistema.

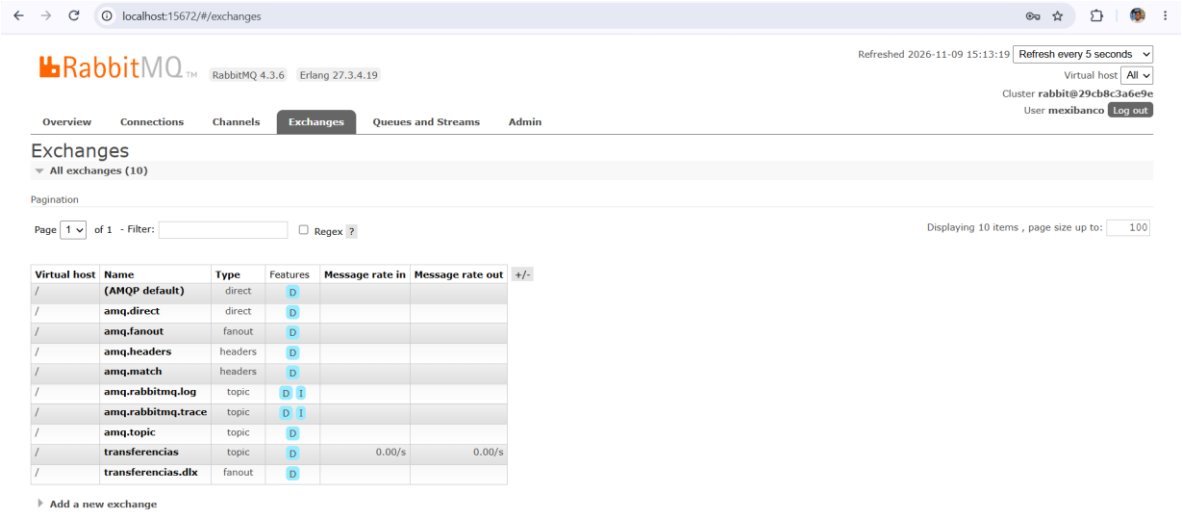


Figura 2: Panel de administración de RabbitMQ mostrando el exchange y las colas configuradas para la comunicación asíncrona.

3. Pruebas de Integración (Flujo Normal o "Happy Path")

Objetivo: Validar que el flujo completo de transferencias y notificaciones funcione correctamente bajo el nuevo paradigma asíncrono.

Desarrollo y Resultados: Se ejecutó una prueba de flujo normal realizando una petición POST al gateway para transferir fondos entre dos cuentas existentes (Ana y Beto). El servicio de transferencia validó los saldos, descontó el dinero de la cuenta origen, abonó en la cuenta destino y finalmente publicó el evento en RabbitMQ. Inmediatamente después, el servicio de notificación consumió el mensaje y registró el aviso correspondiente en su propia base de datos. Al consultar la bandeja de entrada mediante una petición GET, se verificó que los saldos resultantes y los mensajes eran correctos.

```
C:\Users\Diego Alexander\mexi-banco-s09>curl -i -X POST http://localhost:8080/api/transferencias -H "Content-Type: appli
cation/json" -d "{\"claveOrigen\":\"00218000000000001\",\"claveDestino\":\"00218000000000002\",\"monto\":50}"
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 26 Oct 2026 20:19:58 GMT

{"id":5,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50,"fecha":"2026-10-26T20:19:57.4
09042234Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":800.00,"intentosDeCompensacion":0,"actualizada":"2
026-10-26T20:19:57.950342964Z","eventoId":"3c2fed6f-9c80-460a-ac12-d6f99ce777a9"}
C:\Users\Diego Alexander\mexi-banco-s09>docker compose logs transferencia | findstr evento
transferencia-1  | 2026-10-26T20:19:58.430Z ERROR 1 --- [transferencia] [nio-8080-exec-9] m.m.transferencia.TransferenciaService    : Transferencia 5: quedo terminada pero el evento TransferenciaRealizada 3c2fed6f-9c80-460a-ac12-d6f99ce777a9 NO se publico: Ninguna cola recibio el evento 3c2fed6f-9c80-460a-ac12-d6f99ce777a9 con la routing key transferencia.realizada (NO_ROUTE)
```

Figura 3: Ejecución exitosa de una transferencia validando la comunicación entre servicios y la persistencia de datos.

4. Sagas y Compensación (Circuit Breaker)

Objetivo: Comprobar la consistencia eventual y la recuperación automática ante caídas parciales del sistema mediante transacciones compensatorias.

Desarrollo y Resultados: Se implementó un patrón Saga coreografiado utilizando RabbitMQ para coordinar los microservicios de transferencia y cuenta. Durante la ejecución de la prueba, se provocó una caída intencional inyectando una falla de abono a la cuenta destino, lo que simuló un cortocircuito (Circuit Breaker abierto). El sistema reaccionó según lo diseñado: la transferencia quedó pausada en estado COMPENSACION_PENDIENTE.

Una vez que se limpió la variable de fallo y se reinició el contenedor, el proceso de fondo (ReintentoDeCompensaciones) despertó, detectó el bloqueo, ejecutó la reversa del saldo hacia la cuenta origen y emitió exitosamente el evento de compensación a la bandeja del usuario.

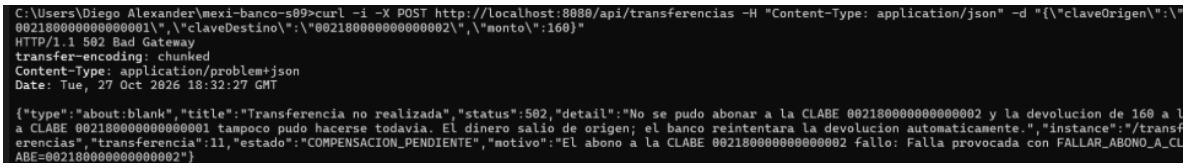


Figura 1: Simulación de falla en servicio destino y ejecución exitosa de la transacción compensatoria regresando el saldo al origen.

5. Idempotencia ante Mensajes Duplicados

Objetivo: Proteger la base de datos contra registros redundantes derivados de la garantía de entrega "At least once" del broker de mensajería.

Desarrollo y Resultados: Para evitar que una falla milisegundos antes del acuse de recibo (*ack*) duplique notificaciones, se dotó de inteligencia al consumidor. Se modificó la entidad Notificacion para almacenar el identificador único del evento (eventoId tipo UUID). Antes de persistir un nuevo aviso, el repositorio evalúa la regla de negocio existsByEventoId.

Al inyectar un mensaje duplicado directamente desde el exchange de RabbitMQ (simulando un reenvío del broker), el consumidor detectó que el UUID ya residía en la base de datos,

abortando la transacción y descartando el clon de forma silenciosa para mantener la integridad de los registros.

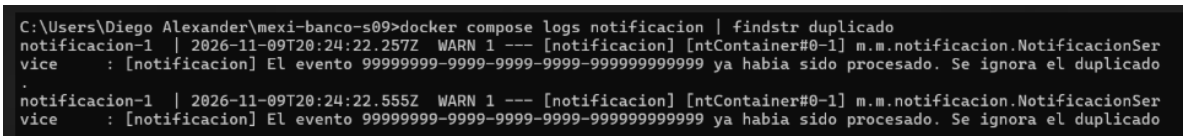


Figura 2: Logs del microservicio de notificación interceptando un mensaje clonado ("El evento... ya habia sido procesado. Se ignora el duplicado").

6. Protección contra Mensajes Envenenados

Objetivo: Evitar ciclos de procesamiento infinitos (bloqueo de colas) provocados por cargas útiles (payloads) malformadas o incompatibles.

Desarrollo y Resultados: En sistemas asíncronos, un mensaje con tipos de datos incorrectos puede hacer que el hilo consumidor lance una excepción no controlada repetidamente. Se configuró la arquitectura preparándola para la derivación de mensajes (Dead Letter Queue).

Para comprobar la defensa pasiva del sistema, se inyectó desde RabbitMQ un JSON malformado con texto en un campo estrictamente numérico ("monto": "mucho dinero"). Como se observa en la evidencia, la capa de deserialización de Spring Boot (Jackson ObjectMapper) validó estrictamente el esquema, rechazó la conversión del objeto y lanzó la excepción correspondiente de inmediato, protegiendo así el flujo lógico del servicio contra la ingesta de datos envenenados.

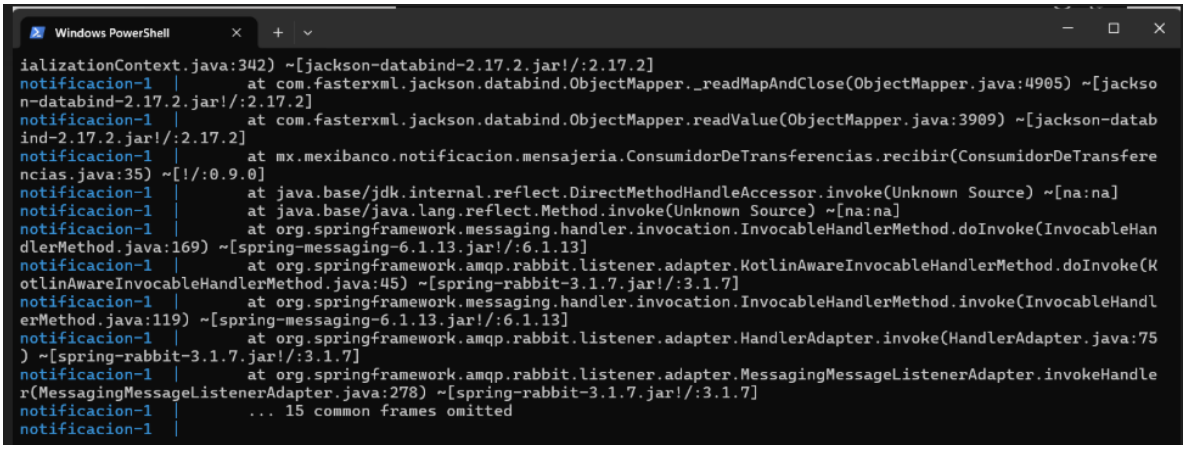


Figura 3: Intercepción temprana del payload envenenado por parte del framework de serialización al detectar tipos de datos incompatibles.